COS AI — Architecture Specification v0.1

Derived from Book 1: Automatic Digital Computation (1953 Symposium) and Book 2: Structure and Interpretation of Computer Programs (SICP)

This is the first architectural synthesis of the two books. It is not a copy of either architecture. Book 1 gives COS lessons about machines, storage, timing, reliability, diagnostics and hardware/software co-design; Book 2 gives lessons about abstraction, languages, data, state, interpreters, compilers and register machines.

COS AI

Cognitive Operating System

Architecture Specification v0.1

Project: SaemLabs
Architecture generation: 0.1
Status: Foundational / pre-v2.2 implementation architecture
Primary implementation languages: C17 → C++ where justified
Design principle: Build from machine reality upward while preserving semantic meaning downward-to-upward.

---

1. Architectural Objective

COS AI is not primarily an application, chatbot, neural network, or conventional operating system.

Its architectural objective is:

«To create a machine-independent cognitive system that can understand an intention, represent its meaning, reason about it, select appropriate computational resources, and eventually execute that intention across heterogeneous machines.»

Therefore COS must distinguish:

1. What is intended
2. What it means
3. What knowledge is required
4. What reasoning is required
5. What computation is required
6. What machine resources are available
7. How the computation is translated to those resources
8. How execution is verified
9. What happened
10. Why it happened

---

2. Fundamental Architectural Law

COS shall preserve the distinction:

MEANING ≠ REPRESENTATION ≠ EXECUTION ≠ HARDWARE

For example:

"solve this differential equation"

must not become permanently tied to:

CPU instruction

or:

GPU kernel

or:

COS-B instruction

Instead:

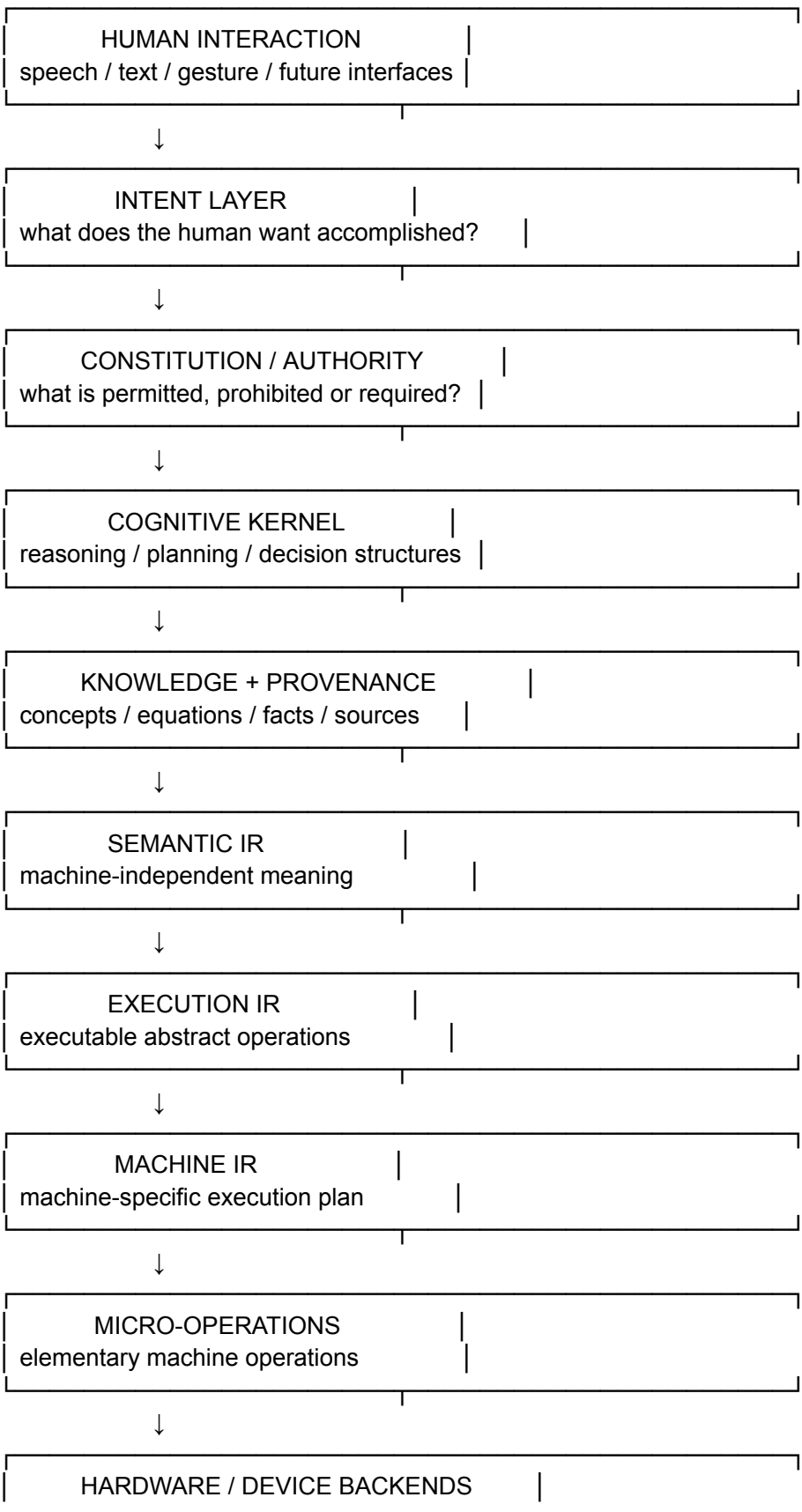
Human Intent
↓
Semantic Representation
↓
Reasoning Plan
↓
Abstract Computation
↓
Execution Representation
↓
Machine Representation
↓
Hardware

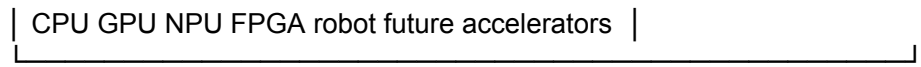
This separation is one of the central architectural consequences of Book 2.

---

3. The COS Layer Model

The first formal COS stack is:


HUMAN INTERACTION
speech / text / gesture / future interfaces
INTENT LAYER
what does the human want accomplished?
CONSTITUTION / AUTHORITY
what is permitted, prohibited or required?
COGNITIVE KERNEL
reasoning / planning / decision structures
KNOWLEDGE + PROVENANCE
concepts / equations / facts / sources
SEMANTIC IR
machine-independent meaning
EXECUTION IR
executable abstract operations
MACHINE IR
machine-specific execution plan
MICRO-OPERATIONS
elementary machine operations
HARDWARE / DEVICE BACKENDS



This is the primary architectural spine of COS.

---

4. Layer 0 — Human Interface

COS must eventually accept:

- natural language
- mathematical notation
- programming languages
- speech
- structured commands
- sensor information
- machine/device information

and eventually potentially:

- vision
- gesture
- physical interaction
- multimodal streams

However, the interface must not directly control hardware.

Instead:

Human communication
↓
Language understanding
↓
Intent representation

The interface is therefore a translator into COS meaning.

---

5. Layer 1 — Intent Representation

Intent answers:

«What is the requested objective?»

Example:

"Find the trajectory of this projectile."

could become:

```
INTENT
{
   objective: SOLVE
   object: PROJECTILE_MOTION
   desired_output: TRAJECTORY
}
```

The intent layer must remain independent of implementation.

It must not contain:

ADD R1,R2

or:

CUDA

or:

AVX

or:

FPGA

Those belong to later layers.

---

6. Layer 2 — Constitution and Authority

COS contains an architectural boundary between:

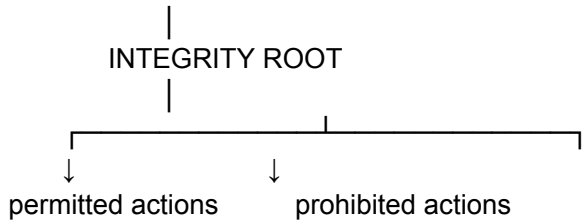
IMMUTABLE / PROTECTED ROOT

and:

MUTABLE COGNITIVE STATE

Conceptually:





The constitutional layer is not ordinary learned knowledge.

It must be:

- versioned
- provenance-linked
- integrity protected
- independently verified
- tamper evident
- fail closed

A failed constitutional integrity check must cause COS to refuse the affected operation, rather than silently operating under an altered constitution.

---

7. Layer 3 — Cognitive Kernel

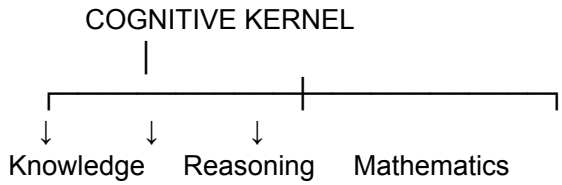
The Cognitive Kernel is the architectural center of COS.

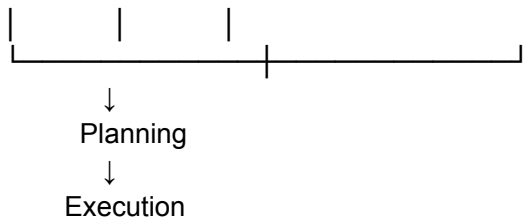
Its initial responsibilities are:

reason
plan
decompose
verify
select representations
select resources
execute cognitive operations
inspect results
recover from failures

The kernel must not itself contain every mathematical or scientific fact.

Instead it accesses specialized systems:





This keeps the kernel small.

---

8. Layer 4 — Knowledge Architecture

COS knowledge is structured rather than treated as one undifferentiated memory.

The fundamental hierarchy is:

SOURCE
  ↓
COURSE
  ↓
LECTURE / DOCUMENT
  ↓
CONCEPT
  ↓
DEFINITION / EQUATION / PRINCIPLE
  ↓
PREREQUISITES
  ↓
RELATIONSHIPS
  ↓
VERIFIED KNOWLEDGE ATOM

A knowledge atom should have, at minimum:

ATOM
 ├── identity
 ├── content
 ├── type
 ├── source
 ├── locator
 ├── provenance
 ├── verification status
 ├── prerequisites
 ├── relationships
 └── representation

This follows the architectural lesson that data representation matters.

Knowledge should therefore be inspectable.

COS must be able to answer:

«Where did this knowledge come from?»

and:

«Why does COS currently trust this knowledge?»

---

9. Knowledge Status

The existing COS knowledge statuses remain:

SOURCE_VERIFIED
DERIVED
UNVERIFIED
REJECTED

Derived knowledge must not silently become source knowledge.

For example:

Source fact
↓
Derivation
↓
Derived result

The provenance chain must remain available.

---

10. Layer 5 — Reasoning

COS reasoning operates on structured representations.

Initial reasoning modes include:

forward reasoning
backward reasoning
dependency traversal
symbolic transformation
proof construction
constraint checking

contradiction detection

The Knowledge Graph provides relationships:

A prerequisite_of B
A supports B
A contradicts B
A derived_from B
A relates_to B

The reasoning engine should generate:

CONCLUSION
+
PROOF / DERIVATION TRACE
+
DEPENDENCIES
+
PROVENANCE

rather than merely producing an answer.

---

11. Layer 6 — Mathematical Kernel

Mathematics is a particularly important test of COS architecture.

The current mathematical progression is:

Exact integers
↓
Rationals
↓
Units / dimensions
↓
Expressions
↓
Algebra
↓
Linear algebra
↓
Tensors
↓
Calculus
↓
Mathematical analysis
↓

ODEs
  ↓
Numerical methods
  ↓
Automatic differentiation
  ↓
Elementary functions

The key architectural distinction is:

EXACT
vs.
APPROXIMATE

COS must never silently convert an exact result into an approximation.

For example:

1/3

must remain an exact rational when exactness is required.

An approximate representation must carry its approximation semantics and, where available, error information.

---

12. Layer 7 — Semantic IR

The Semantic IR represents meaningful operations, not machine instructions.

Example:

```
SOLVE
  problem = differential_equation
  variables = [x, y]
  constraints = [...]
  desired_output = symbolic_solution
```

Another:

```
MATRIX_MULTIPLY
  A = matrix_1
  B = matrix_2
```

Another:

```
DERIVE
```

```
function = f(x)
variable = x
```

The Semantic IR should remain independent of:

- CPU architecture
- instruction set
- GPU architecture
- memory arrangement
- operating system
- programming language syntax

---

13. Layer 8 — Execution IR

Execution IR converts semantic operations into executable abstract operations.

Example:

SEMANTIC

MATRIX_MULTIPLY(A,B)

↓

EXECUTION IR

```
LOAD A
LOAD B
ALLOCATE RESULT
FOR i
   FOR j
      ACCUMULATE
STORE RESULT
VERIFY
```

The Execution IR therefore bridges:

meaning

and:

machine execution

This is the layer where the current COS CIR belongs.

---

14. Layer 9 — Machine IR

Machine IR answers:

«How should this computation be performed on this particular machine?»

It incorporates:

register availability
memory hierarchy
instruction set
parallelism
vector units
accelerators
latency
bandwidth
device capabilities
resource limits

This is directly influenced by Book 1.

The same abstract operation may therefore produce different Machine IR on different machines.

Example:

Semantic:

VECTOR_DOT(A,B)

Machine A:

scalar CPU operations

Machine B:

SIMD vector instructions

Machine C:

GPU kernel

Machine D:

FPGA dataflow

The semantic operation remains the same.

---

15. Layer 10 — Micro-Operations

Complex machine operations may be decomposed into elementary operations.

Conceptually:

MACHINE INSTRUCTION
↓
micro-operation sequence
↓
register transfer
↓
arithmetic / logic
↓
memory access

This creates a conceptual bridge from:

COS computation

to:

physical machine behavior

This layer is essential for the long-term goal of understanding and eventually optimizing hardware from low-level principles.

---

16. Hardware Abstraction

COS must not assume one permanent hardware architecture.

The Machine Capability Model should describe available resources.

Example:

MACHINE
├── CPU
│ ├── cores
│ ├── registers
│ ├── ISA
│ └── cache hierarchy
│
├── GPU

```
│    ├── compute units
│    ├── memory
│    └── parallel capabilities
│
├── NPU
│
├── FPGA
│
├── MEMORY
│
├── NETWORK
│
└── DEVICES
```

A future quantum system can appear as another backend:

```
QUANTUM BACKEND
 ├── qubits
 ├── gates
 ├── measurement
 ├── topology
 ├── coherence constraints
 └── error model
```

COS should therefore learn machine capabilities, not merely machine names.

---

17. Storage Is a Computational Resource

One of the strongest lessons from Book 1 is:

«Storage is not merely a passive container.»

COS therefore distinguishes:

storage capacity
storage latency
bandwidth
access pattern
persistence
locality
concurrency
reliability

The execution system must be aware that:

COMPUTATION COST

=
ARITHMETIC COST
+
DATA MOVEMENT COST
+
SYNCHRONIZATION COST
+
I/O COST

The exact cost model will evolve.

---

18. Time Is an Architectural Resource

COS must model time explicitly.

At minimum:

timestamp
duration
deadline
latency
ordering
causality
synchronization

For machines:

memory delay
I/O delay
network latency
device response time
computation time

For cognition:

event
→ observation
→ interpretation
→ decision
→ action
→ feedback

This is necessary for robotics, distributed systems, sensors and real-world interaction.

---

19. Streams

COS cannot be designed only around static files.

The world produces continuous information:

sensor
sensor
sensor
sensor
...

Therefore COS needs a stream abstraction:

STREAM
=
time-indexed sequence of events

Examples:

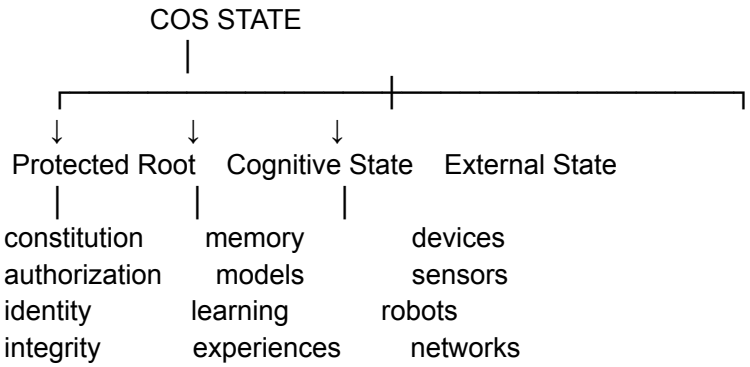
camera stream
audio stream
temperature stream
network stream
robot telemetry
human interaction

The cognitive kernel can consume these streams incrementally.

---

20. State Architecture

State is divided into classes.



Cognitive state may change.

Constitutional root does not change through ordinary learning.

This separation prevents:

learning

from automatically becoming:

constitutional authority

---

21. Memory Classes

COS memory should eventually distinguish:

1. Constitutional memory
2. Identity/integrity memory
3. Verified knowledge
4. Derived knowledge
5. Working memory
6. Temporary computation
7. Learned models
8. Device state
9. Event/experience logs
10. Provenance/audit records
11. Caches

Different memory classes require different:

- lifetimes
- permissions
- persistence
- verification
- garbage-collection rules

---

22. Language Architecture

COS should have multiple languages.

Not one language.

Proposed stack:

Human Language
↓

Intent Language
↓
COS Semantic Language
↓
COS Execution Language
↓
COS-B
↓
COS Bytecode
↓
Machine IR
↓
Machine instructions

COS-B therefore becomes one language in a larger language family.

It should not be forced to represent every cognitive abstraction.

---

23. COS-B

COS-B is a low-level programmable language influenced by the historical study of early machines and by the register-machine concepts of SICP.

Its responsibilities are:

control
data movement
arithmetic
branching
memory access
I/O
machine-level execution

COS-B should remain:

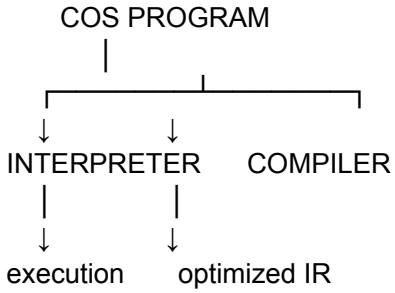
- small
- deterministic where possible
- inspectable
- compilable
- simulatable
- formally representable

The existing COS-B VM therefore becomes a foundation rather than the final COS language.

---

24. Interpreter and Compiler

COS should support both.



Interpreter:

- useful for experimentation
- debugging
- education
- semantic validation

Compiler:

- optimization
- deployment
- specialization
- hardware acceleration

Both should ultimately preserve the same semantics.

---

25. Self-Description

A major long-term property of COS is that the system should be able to represent descriptions of:

its own language
its own IR
its own machine model
its own programs
its own knowledge structures

Conceptually:

PROGRAM
  can be represented as DATA

LANGUAGE

can be represented as DATA

MACHINE
can be represented as DATA

KNOWLEDGE
can be represented as DATA

This is important for eventual self-hosting.

---

26. Machine Simulation

COS should be able to simulate machines before controlling them.

Architecture:

Machine Description
↓
Machine Simulator
↓
Test Program
↓
Execution Trace
↓
Verification

This creates a safe bridge between:

reasoning

and:

physical execution

It also supports future hardware design.

---

27. Diagnostics

Diagnostics are architectural, not an afterthought.

Every important COS subsystem should expose:

status
trace

errors
invariants
resource usage
provenance
execution history

A failed computation should ideally produce:

WHAT failed?
WHERE?
WHEN?
WHY?
WITH WHAT INPUT?
USING WHICH VERSION?
USING WHICH KNOWLEDGE?
ON WHICH MACHINE?

---

28. Observability

COS execution should be inspectable at multiple levels:

Intent
↓
Semantic IR
↓
Execution IR
↓
Machine IR
↓
Instructions
↓
Micro-operations
↓
Hardware event

A high-level error should therefore eventually be traceable downward.

Likewise a hardware failure should be traceable upward.

This gives COS a bidirectional debugging path:

MEANING → MACHINE
MACHINE → MEANING

---

29. Reliability

COS must assume that hardware eventually fails.

Therefore:

failure detection
↓
diagnosis
↓
isolation
↓
recovery
↓
verification
↓
resume

Possible future mechanisms:

redundancy
checkpointing
replication
self-test
fault isolation
alternative backend
rollback
verified recovery

Self-repair must therefore be treated as an engineering subsystem, not as unrestricted self-modification.

---

30. Physical Modularity

COS hardware interfaces should be modular.

A device should ideally expose:

capabilities
identity
interface
status
diagnostics
control operations
failure information

rather than requiring the cognitive kernel to know every physical detail.

Thus:

COS
↓
Device abstraction
↓
Driver/backend
↓
Physical device

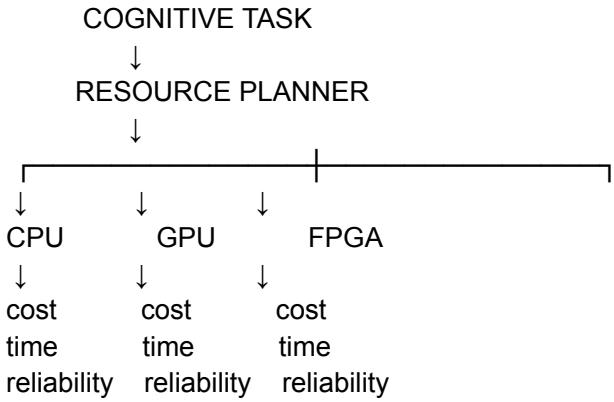
---

31. Resource Manager

Eventually COS requires a resource manager capable of deciding:

Which computation?
Which machine?
Which memory?
Which accelerator?
How much energy?
How much time?
What priority?
What reliability requirement?

Conceptually:



This must be driven by machine capabilities and task requirements rather than hard-coded device names.

---

32. Generic Operations

The same abstract operation should support multiple representations.

Example:

ADD

may operate on:

integer
rational
real approximation
complex number
vector
matrix
tensor
symbolic expression
interval

The semantic operation remains:

ADD

while its representation changes.

This is a direct architectural consequence of abstraction and generic operations.

---

33. Representation Independence

COS should preserve:

ABSTRACT OBJECT
↓
REPRESENTATION

and permit multiple representations.

For example:

Matrix
├── dense matrix
├── sparse matrix
├── symbolic matrix
├── distributed matrix
└── accelerator representation

The user should not need to know which representation COS chooses.

---

34. Execution Pipeline

The canonical execution pipeline becomes:

1. RECEIVE INTENT

2. PARSE / INTERPRET

3. AUTHORIZE

4. CONSTRUCT SEMANTIC REPRESENTATION

5. RETRIEVE REQUIRED KNOWLEDGE

6. BUILD REASONING PLAN

7. VERIFY PREREQUISITES

8. CONSTRUCT EXECUTION IR

9. DISCOVER MACHINE CAPABILITIES

10. SELECT EXECUTION STRATEGY

11. GENERATE MACHINE IR

12. EXECUTE

13. OBSERVE

14. VERIFY RESULT

15. RECORD PROVENANCE

16. RETURN RESULT

---

35. Example

Human:

"Calculate the trajectory of this projectile."

COS:

HUMAN INPUT
↓
INTENT
↓
PROJECTILE_TRAJECTORY
↓
retrieve:
Newtonian mechanics
kinematics
coordinate definitions
initial conditions
↓
REASON
↓
derive equations
↓
SEMANTIC IR
↓
solve differential/equational system
↓
EXECUTION IR
↓
select exact/symbolic or numerical method
↓
MACHINE IR
↓
CPU / GPU / other backend
↓
EXECUTE
↓
VERIFY
↓
RESULT + DERIVATION + PROVENANCE

The machine is therefore the executor of a cognitive plan, not the source of the meaning of that plan.

---

36. Concurrency

COS must eventually support multiple simultaneous processes:

sensor acquisition
+
reasoning
+

communication
+
robot control
+
memory operations
+
learning

Therefore the architecture needs:

events
tasks
queues
locks
ownership
synchronization
causal ordering
deadlines
interrupts

Concurrency should be explicit rather than hidden.

---

37. I/O

Input/output must be treated as computationally significant.

COS therefore distinguishes:

computation
communication
storage
I/O

and permits overlap:

COMPUTATION


I/O


COMMUNICATION


rather than assuming the machine must always perform:

compute → wait → I/O → compute

---

38. Security Boundary

Every transition toward physical effect must pass through explicit authority.

COGNITIVE DECISION
↓
AUTHORIZATION
↓
CAPABILITY CHECK
↓
RESOURCE CHECK
↓
EXECUTION

Possessing a capability does not automatically authorize its use.

This becomes especially important for:

robots
networks
physical actuators
external systems
self-modification

---

39. Self-Modification

COS may eventually need to modify:

programs
models
data structures
execution strategies
drivers
hardware configurations

But self-modification should follow:

PROPOSE
↓
ANALYZE
↓
VERIFY

↓
SANDBOX
↓
TEST
↓
COMPARE
↓
AUTHORIZE
↓
DEPLOY
↓
MONITOR
↓
ROLLBACK if required

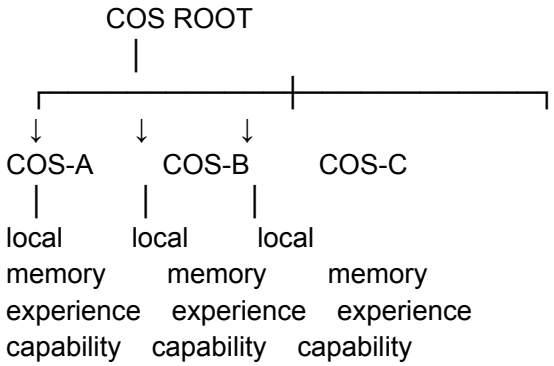
The protected constitutional root is outside ordinary self-modification.

---

40. Child / Instance Architecture

Future COS instances may be created from a protected common root.

Conceptually:



Shared:

constitutional identity
core semantics
foundational architecture

Potentially different:

experience
knowledge extensions
hardware
local memory

specialized capabilities

This remains a future architectural concept, not a current implementation claim.

---

41. Book 1 → COS Principles

The first book establishes the following engineering principles for COS:

P01 Storage affects computation.

P02 Timing is a computational resource.

P03 Hardware and software must be co-designed.

P04 Complex instructions can be constructed from elementary operations.

P05 Abstraction must retain traceability.

P06 I/O can dominate computation.

P07 Memory hierarchy must be architecturally visible.

P08 Diagnostics belong inside the architecture.

P09 Reliability must be measured.

P10 Physical modularity improves maintainability.

P11 Simplicity can be more valuable than raw complexity.

P12 Machines should evolve through measured engineering feedback.

---

42. Book 2 → COS Principles

SICP establishes complementary principles:

S01 Abstraction is a primary engineering tool.

S02 Procedures should be first-class computational objects.

S03 Data abstraction separates meaning from representation.

S04 Generic operations allow multiple representations.

S05 State must be explicitly modeled.

S06 Time and concurrency require explicit architectural treatment.

S07 Languages can be represented and interpreted as data.

S08 Evaluators are themselves computational systems.

S09 Logic and symbolic reasoning can be expressed computationally.

S10 Register machines provide a bridge between abstractions and hardware.

S11 Interpreters and compilers can coexist.

S12 Memory management is part of language/runtime architecture.

---

43. Combined COS Principle

The two books together lead to a stronger principle:

«Every important abstraction in COS must have a path to execution, and every important execution event must retain a path back to its abstraction.»

Therefore:

ABSTRACTION
↓
IMPLEMENTATION
↓
EXECUTION
↓
OBSERVATION
↓
TRACEABILITY
↓
ABSTRACTION

This is one of the most important architectural laws of COS v0.1.

---

44. Current COS Implementation Mapping

The existing implementation already contains the beginning of this architecture:

COS-B

↓
Assembler
↓
COS Bytecode
↓
COS Execution IR
↓
COS Semantic IR
↓
Knowledge + Provenance
↓
Verification / Adjudication
↓
Knowledge Graph
↓
Symbolic Reasoning
↓
Exact Mathematics
↓
Numerical Mathematics
↓
Automatic Differentiation
↓
Elementary Functions

Therefore the existing work is not discarded.

Instead, v0.1 places it inside the larger architecture.

---

45. What Is NOT Yet Implemented

The following are architectural targets, not current capabilities:

natural-language cognitive kernel
multimodal perception
general planner
full machine capability discovery
machine IR
micro-operation hardware model
GPU backend
NPU backend
FPGA backend
robot backend
quantum backend
distributed cognition
self-hosting compiler

self-repair
autonomous hardware design
child-instance management
full constitutional execution layer

These must not be claimed as implemented merely because the architecture defines them.

---

46. Immediate Architecture Gap

The largest gap between the current implementation and COS v0.1 is:

CURRENT:

COS-B → CIR → SIR → Knowledge/Reasoning/Math

TARGET:

Human Intent
↓
Constitution
↓
Cognition
↓
Knowledge / Reasoning
↓
Semantic IR
↓
Execution IR
↓
Machine IR
↓
Micro-operations
↓
Hardware

The next architectural work should therefore not immediately jump to another mathematical library.

The missing bridge must first be designed.

---

47. Proposed Next Kernel Boundary

The next major boundary should be:

COS Semantic IR
↓
COS Execution Planner
↓
Machine Capability Model
↓
Machine IR

This is more important architecturally than simply adding more mathematical functions.

---

48. COS v0.1 Module Map

```
cos/
│
├── constitution/
│
├── identity/
│
├── intent/
│
├── cognition/
│
├── knowledge/
│
├── provenance/
│
├── verification/
│
├── reasoning/
│
├── mathematics/
│
├── language/
│   ├── cosb/
│   ├── assembler/
│   └── compiler/
│
├── ir/
│   ├── semantic/
│   ├── execution/
│   └── machine/
│
├── runtime/
│
```

```
│   ├── scheduler/
│   ├── memory/
│   ├── events/
│   └── streams/
│
├── machine/
│
│   ├── capability/
│   ├── simulator/
│   └── backend/
│
├── devices/
│
├── diagnostics/
│
├── observability/
│
└── tests/
```

This is an architectural organization, not a requirement to implement every directory immediately.

---

49. Foundational Invariants

COS v0.1 should preserve these invariants:

I1 — Meaning Preservation

A translation must preserve the semantics of the source representation.

I2 — Provenance Preservation

A derived result must retain its derivation/source chain.

I3 — Authorization Before Effect

Physical/external effects require authorization before execution.

I4 — Constitutional Integrity

A failed constitutional integrity check must fail closed.

I5 — Exactness Preservation

Exact computations must not silently become approximate.

I6 — Representation Independence

Changing representation must not change abstract meaning.

I7 — Traceability

Important operations must remain traceable across architectural layers.

I8 — Verification

Important results should be independently checkable where feasible.

I9 — Capability Awareness

Execution must depend on known machine capabilities.

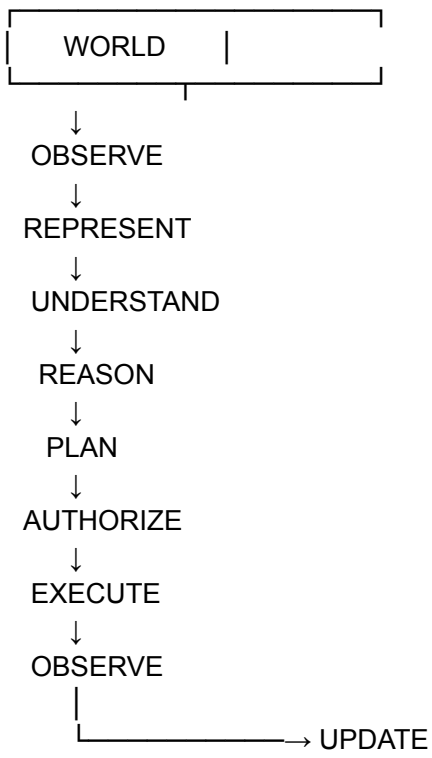
I10 — Observability

Execution must expose sufficient information for diagnosis and recovery.

---

50. The Central COS Loop

The eventual cognitive operating loop is:



But COS must not update its constitutional root through this loop.

Instead:

WORLD
↓
EXPERIENCE
↓
COGNITIVE STATE

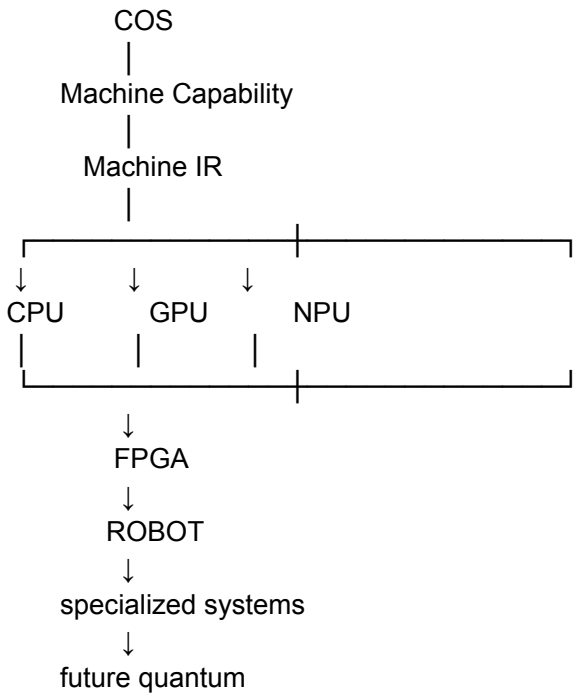
while:

CONSTITUTION
↓
remains protected

---

51. Long-Term Hardware Vision

The architectural direction is:



The architecture should therefore survive replacement of individual hardware generations.

The machine changes.

The cognitive abstractions should survive.

---

52. Design Philosophy

COS should follow:

SMALL CORE
+
STRONG ABSTRACTIONS
+
EXPLICIT REPRESENTATIONS
+
VERIFIABLE KNOWLEDGE
+
TRACEABLE EXECUTION
+
MODULAR HARDWARE
+
MEASURED EVOLUTION

Not:

HUGE MONOLITH
+
HIDDEN STATE
+
OPAQUE EXECUTION
+
UNTRACEABLE LEARNING

---

53. What We Learned From the Two Books

Book 1 teaches COS:

«A computer is a physical system whose storage, timing, I/O, reliability and maintenance determine what computation is practically possible.»

Book 2 teaches COS:

«A computer is also a hierarchy of abstractions in which procedures, data, state, languages, interpreters and machine descriptions can themselves become computational objects.»

Together:

«COS must be both a computational organism and a system of abstractions capable of describing, reasoning about, simulating and controlling computational organisms.»

That is the architectural foundation.

---

54. Status of Architecture v0.1

Established from Books 1–2

✓ abstraction layers
✓ representation independence
✓ structured data
✓ provenance/traceability
✓ state separation
✓ streams/time
✓ interpreters
✓ compilers
✓ register-machine bridge
✓ storage awareness
✓ timing awareness
✓ diagnostics
✓ reliability
✓ modular hardware
✓ machine/software co-design

COS-specific design decisions

✓ Constitution as protected root
✓ Semantic IR
✓ Execution IR
✓ Machine IR
✓ Knowledge/provenance architecture
✓ capability-based hardware abstraction
✓ exact/approximate mathematical distinction
✓ future heterogeneous backend architecture
✓ protected child-instance concept

Future research

? cognitive architecture
? learning architecture
? perception
? machine self-model
? hardware synthesis
? quantum execution
? distributed COS
? autonomous repair
? self-hosting

These remain open research questions.

---

55. The Next Step

Do not immediately add another large feature.

The architecture now suggests the following sequence:

BOOK 1
 ↓
BOOK 2
 ↓
COS ARCHITECTURE v0.1 ← CURRENT
 ↓
BOOK 3
 ↓
ARCHITECTURE REVISION
 ↓
BOOK 4
 ↓
ARCHITECTURE REVISION
 ↓
...
 ↓
COS ARCHITECTURE v0.2

The approximately forty books should therefore change the architecture progressively.

We should not freeze COS after two books.

The correct philosophy is:

«Read → extract principles → test against existing architecture → revise architecture → implement only when the architectural reason is clear.»

---

56. Final Architectural Statement

COS is therefore defined, at this stage, as:

«A layered, machine-independent cognitive architecture in which human intent is transformed into verified semantic representations, reasoned over using structured knowledge and mathematics, translated into executable representations, mapped onto machine capabilities, executed through heterogeneous hardware, and observed through a

traceable verification and diagnostic system—while maintaining a protected constitutional root and a strict separation between meaning, representation, state and physical execution.»

COS Architecture v0.1 — frozen as a study baseline, not as the final architecture.The important consequence is that we should not yet rush into v2.2 code. Books 1–2 have exposed a missing architectural layer: the path from COS Semantic IR → execution planning → machine capability model → Machine IR.

That is the bridge I would design next after Book 3, rather than guessing its final form now.